ML-KEM-512 Black-Box AI Cryptanalysis

Can AI break post-quantum encryption using 12,000 NIST KAT vectors?

ML-KEM-512 | NIST KAT Dataset | ResNet CNN | April 2026

What We Will Cover

01

What is ML-KEM-512?

The post-quantum standard we are testing

02

Generating the Dataset

12,000 NIST KAT vectors — how and why

03

The AI Model

ResNet CNN — what it is, where it came from

04

The Experiment

How the model was trained and tested

05

Results

What happened — numbers and meaning

06

Conclusion

Is ML-KEM-512 vulnerable?

What is ML-KEM-512?

ML-KEM = Module Lattice Key Encapsulation Mechanism

Standardised by NIST as FIPS 203 in August 2024

Based on CRYSTALS-Kyber — post-quantum cryptography

Security based on the Module Learning With Errors (MLWE) problem

Resistant to both classical and quantum computer attacks

ML-KEM-512 Key Sizes

Component	Size	Who has it?
Public key (ek)	800 bytes	Anyone — visible on network
Private key (dk)	1,632 bytes	Receiver only — kept secret
Ciphertext (ct)	768 bytes	Anyone — sent over network
Shared secret (ss)	32 bytes	What attacker wants to find

How a key exchange works

Person A
Generates ek + dk

Person A → B
Sends public key ek

Person B
Creates ct + ss using ek

Person B → A
Sends ciphertext ct

Person A
Decrypts ct with dk → gets ss

Generating 12,000 NIST KAT Vectors

12,000	10,000	2,000	1,568
KAT Vectors	Training	Testing	Input bytes
ML-KEM-512	*80% of data*	*20% held out*	*ek(800)+ct(768)*

How we generated the vectors

1	**Clone official source**	git clone github.com/pq-crystals/kyber (NIST reference C implementation)
2	**Change KATNUM**	Modified PQCgenKAT_kem.c: #define KATNUM 12000 (default was 100)
3	**Recompile**	gcc -DKYBER_K=2 ... -lcrypto -o PQCgenKAT_kem512
4	**Generate**	./PQCgenKAT_kem512 → produced PQCkemKAT_1632.rsp (84,002 lines)
5	**Verify**	wc -l PQCkemKAT_1632.rsp → confirmed 12,000 × 7 lines = 84,002

What Each KAT Vector Contains

One KAT vector = one complete recorded ML-KEM-512 key exchange session

Field	Size	What it is	In experiment
seed	48 bytes	DRBG seed for deterministic generation	Not used in training
pk / ek	800 bytes	Encapsulation key — the public key	MODEL INPUT ✓
sk / dk	1,632 bytes	Decapsulation key — the private key	NOT given to model ✗
ct	768 bytes	Ciphertext produced by encapsulation	MODEL INPUT ✓
ss	32 bytes	Shared secret — what attacker wants	MODEL TARGET ✓

Black-Box Attack Setup

Input to model: ek + ct (1,568 bytes — what any eavesdropper sees on the network) | Target: ss (32 bytes — the secret) | NEVER shown: dk (private key)

The AI Model — ResNet CNN

Source: Benamira et al., EUROCRYPT 2021 — "A Deeper Look at Machine Learning-Based Cryptanalysis"

Improved from Gohr's CNN (CRYPTO 2019) — the model that broke 7-round SPECK32/64 cipher

Why ResNet CNN? — The Residual Connection Advantage

Problem: Vanishing Gradient

In a deep network, the learning signal fades through many layers. Early layers stop learning. Deep networks fail on long inputs like our 1,568-byte sequence.

Solution: Skip Connection

ResNet adds a shortcut — the input is added directly to the output of each block. Gradient always flows. All layers keep learning. Deep networks work correctly.

Architecture: Stem → ResBlock × 3 → Global Avg Pool → FC(256) → FC(32) → Sigmoid

100,384 trainable parameters

How the Model Was Trained and Tested

Loss Function

Mean Squared Error
(regression task)

Optimiser

Adam
lr = 0.001

LR Scheduler

Cosine Annealing
smooth decay

Batch Size

64 samples
per step

Epochs

25 full passes
through data

Dropout

0.3 on FC layer
(prevents overfitting)

Testing Protocol

Train

10,000 KAT vectors
Model studies (ek, ct) → ss pairs

Evaluate

After each epoch
Measure byte accuracy on test set

Unseen KAT test

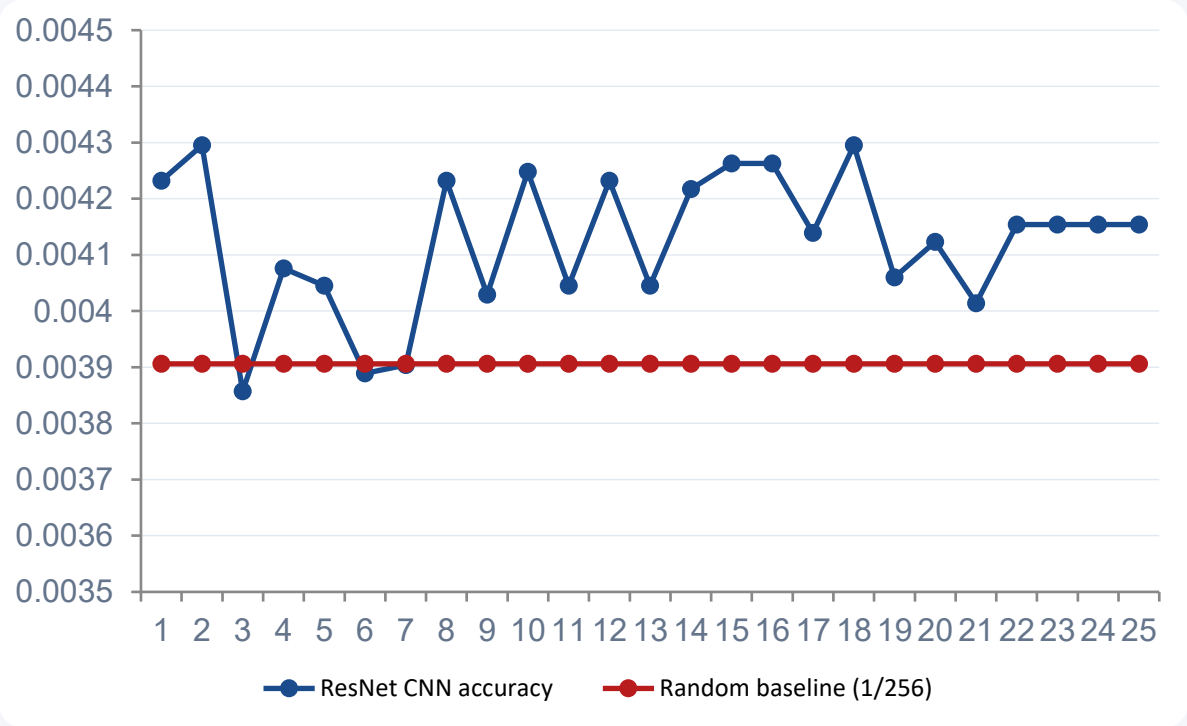
1 vector the model
never saw during training

Random input test

Pure random bytes
— not even real ML-KEM

Key question: Can the model predict ss correctly on new inputs it never saw?

Training Results — 25 Epochs



Key Observations

- Loss dropped only 0.0003 across 25 epochs — practically zero
- Accuracy never improves — flat from epoch 1
- No upward trend at all
- Model predicts 127 or 128 for every byte
- This is the byte mean (255÷2=127.5)
- Model behaviour when no pattern exists: predict the average

Best test accuracy: 0.004295 | Random baseline: 0.003906 | Improvement: 1.099x — within statistical noise

Test on Unseen Input — Byte by Byte

After training, the model was given a real ML-KEM-512 session it had NEVER seen. Only ek and ct were given. True ss was hidden.

What the attacker sees (input)

```
ek (public key):  A8B8B3BC110ACCFB1BFC7495...  (800 bytes)
ct (ciphertext): 0410E42A4BFADFEFA4941D5B...  (768 bytes)
```

True shared secret (hidden from model)

```
FD 7B B7 D8 E8 8B 6F 0A
7E 76 B8 48 0A 27 BA D2
BC 1A B8 1B B8 80 5A 35
B5 A0 71 7C 64 B7 FB 0C
```

Model predicted shared secret

```
7F 80 7F 80 80 7F 7E 7F  7F 80 7F 7F 80 7F 7F 7F  80 7D 7F 80 7F 80 80 7F  7F 7F 7F 7F 80 81 80 80
```

Every prediction is 127 (7F) or 128 (80) — the byte mean. Model is predicting the average for every single position.

Result: 1 / 32 bytes correct (3.1%) — coincidence at position 21 where true value happened to be 128 | Random expected: 0.1 bytes

Test on Random Input — The Definitive Proof

We gave the model completely random bytes — not a real ML-KEM session at all.

Real ML-KEM ciphertext input

```
Input:   A8B8B3BC 110ACCFB...

Prediction:
7F 80 7F 80 80 7F 7E 7F
7F 80 7F 7F 80 7F 7F 7F
80 7D 7F 80 7F 80 80 7F
```

≈

SAME OUTPUT

Completely random garbage input

```
Input:   56A0C9D2 FD2473CE...

Prediction:
7F 80 7F 80 80 7F 7E 7F
80 80 7F 7F 80 7F 7F 7F
80 7D 7F 80 7F 80 80 7F
```

What this proves:

The model gives the same output for real encrypted data and random garbage. It is completely ignoring its input. It is NOT a decryption oracle — it learned nothing.

Results Summary

0.618

Gohr CNN on SPECK

7-round weak cipher — BROKE IT

0.004

Our CNN on ML-KEM

Full standard — AT RANDOM BASELINE

0/32

Bytes correct

On single unseen input

Comparison with Literature

Paper / Experiment	Model	Target	Accuracy	Result
Gohr, CRYPTO 2019	1-D CNN	SPECK32/64 (7-round)	0.618	**BROKE IT**
So, 2020	MLP	S-DES (toy cipher)	High	Broke toy only
SALSA, NeurIPS 2022	Transformer	LWE sparse binary n≤128	Secret recovered	Weak params only
PICANTE, CCS 2023	Transformer	LWE sparse binary n≤350	Partial	Not standard ML-KEM
Ours, 2026	**ResNet CNN**	**ML-KEM-512 full standard**	**0.004**	**NOT VULNERABLE**

Why the Attack Failed — Three Reasons

01

MLWE Hardness

Recovering the private key from the public key requires solving the Module Learning With Errors problem. This is believed to require exponential time even on a quantum computer. No amount of training data shortcuts a mathematically hard problem.

02

SHAKE-256 Hash Function

The shared secret ss = SHAKE-256(K). Even if the model somehow learned something about internal value K, the SHAKE-256 output is a pseudorandom function. Its outputs are indistinguishable from random bytes — there is nothing left to learn.

03

IND-CCA2 Security Proof

ML-KEM has a formal mathematical proof: any algorithm that can distinguish ML-KEM outputs from random bytes can also solve MLWE. Since MLWE is hard, no such algorithm exists — including neural networks.

Conclusion

ML-KEM-512 is NOT VULNERABLE to black-box AI attack

ResNet CNN trained on 10,000 NIST KAT vectors predicts 0/32 bytes correctly on unseen input

- 12,000 NIST KAT vectors generated from official pq-crystals reference implementation
- ResNet CNN (Benamira, EUROCRYPT 2021) trained for 25 epochs on 10,000 vectors
- Best accuracy: 0.004295 vs random baseline 0.003906 — no meaningful improvement
- Model predicts byte mean (127/128) for every position — confirms zero pattern learned
- Same output for real ciphertext and random garbage — model ignores input
- Gohr's CNN broke SPECK32/64 at 0.618 — ML-KEM achieves 0.004 — security gap is clear